Micromouse Task 1: Map and Position

ENCS4380 Interfacing Techniques · Birzeit University · First semester 2026/27 · Team T13 · 11 November 2026

The Task 1 solver maps every reachable cell, and its map dump matches the simulator maze exactly on the 4×4, 8×8 and 16×16 practice mazes: 0 wrong sides out of 64, 256 and 1,024.

1. Requirements checklist

All five Task 1 requirements are met in solver.c, and the required evidence (a map dump matching the simulator maze) passes on three mazes.

#	Requirement	Where it is met in solver.c	Evidence
1	Each cell's walls in 4 bits, written with MM_WALL(dir), no hard-coded numbers	g_walls and g_known (one uint8_t per cell); map_init(), map_set_side()	Section 2.1; the file contains no numeric wall values
2	Every wall recorded on both cells that share it	map_set_side() writes the cell and the neighbour's opposite side	Section 2.2; 0 wrong sides in every dump
3	(x, y) and heading tracked after every move	turn_to(), forward_one(); relative-to-absolute walls in sense_cell()	Section 2.3; mouse labelled every cell with its coordinates and logged its return to (0,0)
4	Maze size and goal read from mm_maze()	maze_init()	Section 3.3: the same code ran 4×4, 8×8 and 16×16, and reported goal (6,1) from the goal file
5	Map dump: mm_map_begin(), mm_map_cell() for each cell, mm_map_end()	report_map()	Section 3.2: dumps of n=16, 64 and 256 cells (4×4, 8×8, 16×16)
E	A map dump that matches the simulator maze	—	Section 3.2: PASS on practice4_1, practice8_offcentre and practice16_1

2. Implementation

The solver keeps two 4-bit maps per cell and its own pose, updates both after every HAL call that succeeds, and walks the maze depth-first so that every reachable cell is seen.

2.1 Wall storage: 4 bits per cell, MM_WALL(dir) only

Each cell has one uint8_t in each of two maps, indexed [x][y]. Only MM_WALL(dir) is used to set, clear or test a bit, because the bit order changes every semester (this semester N=0x8, E=0x4, S=0x2, W=0x1).

```
static uint8_t g_walls[MM_MAX_SIZE][MM_MAX_SIZE];  /* MM_WALL bits: wall here     */
static uint8_t g_known[MM_MAX_SIZE][MM_MAX_SIZE];  /* MM_WALL bits: side observed */

g_walls[x][y] |= MM_WALL(d);              /* set a wall   */
g_walls[x][y] &= (uint8_t)~MM_WALL(d);    /* clear a wall */
if (g_walls[x][y] & MM_WALL(d)) { ... }   /* test a wall  */
```

g_known separates "seen and open" (dumped as 0) from "never seen" (dumped as ?). The outer wall is set in g_walls at start-up but enters g_known only once it is actually seen.

2.2 Every wall on both cells

The east side of (x, y) is the west side of (x+1, y). map_set_side() always writes the cell and its neighbour's opposite side, (d + 2) mod 4, skipping neighbours outside the maze.

```
g_known[x][y] |= MM_WALL(d);                        /* this cell        */
if (in_maze(nx, ny)) {
    int back = dir_back(d);                          /* (d + 2) % 4      */
    g_known[nx][ny] |= MM_WALL(back);                /* the neighbour    */
    if (wall) g_walls[nx][ny] |= MM_WALL(back);
    else      g_walls[nx][ny] &= (uint8_t)~MM_WALL(back);
}
```

2.3 Position and heading

Cell (0,0) is the south-west start; x grows east, y grows north; the mouse starts facing north. The HAL does not track the pose, so the solver updates it only after the HAL returns MM_OK:

- Turn: heading = target direction, after mm_turn(+1 right, −1 left, 2 around) succeeds.
- Forward: x += dx, y += dy after mm_forward(1) succeeds, with N (0,+1), E (+1,0), S (0,−1), W (−1,0).

mm_sense() is relative to the mouse. With heading h, left is (h + 3) mod 4, front is h and right is (h + 1) mod 4. The log prints each conversion, for example:

```
(2,3) facing E: L=1 F=0 R=1  ->  N=1 E=0 S=1
```

2.4 Maze size and goal from mm_maze()

maze_init() reads width, height and the goal block once from mm_maze() and checks them against MM_MAX_SIZE. The arrays are sized MM_MAX_SIZE, but every loop runs to g_w and g_h, so no maze size or goal is written as a constant.

2.5 Map dump

report_map() calls mm_map_begin(), then mm_map_cell(x, y, g_walls[x][y], g_known[x][y]) for every cell, then mm_map_end(). Each cell becomes one line "@C x y N E S W" with 1 (wall), 0 (open) or ? (not seen), followed by "@END n=<cells> crc=<CRC-16>".

2.6 Visiting every cell

To fill the map, the mouse walks depth-first. It enters any open neighbour not visited yet and stores the way back in g_back[x][y]. With none left, it drives one cell back. When it is back at the start with nothing left, every reachable cell has been seen. A last reading at (0,0) sees the start cell's south wall, the only side not in view earlier.

3. Evidence

On all three practice mazes the map dump matches the simulator maze side for side, with every side of every cell known.

3.1 Simulator run

On practice8_offcentre (8×8) the mouse visited all 64 cells, labelled each with its coordinates, drew every wall it found, and finished back at (0,0). The run output ends with @END n=64 crc=8106.

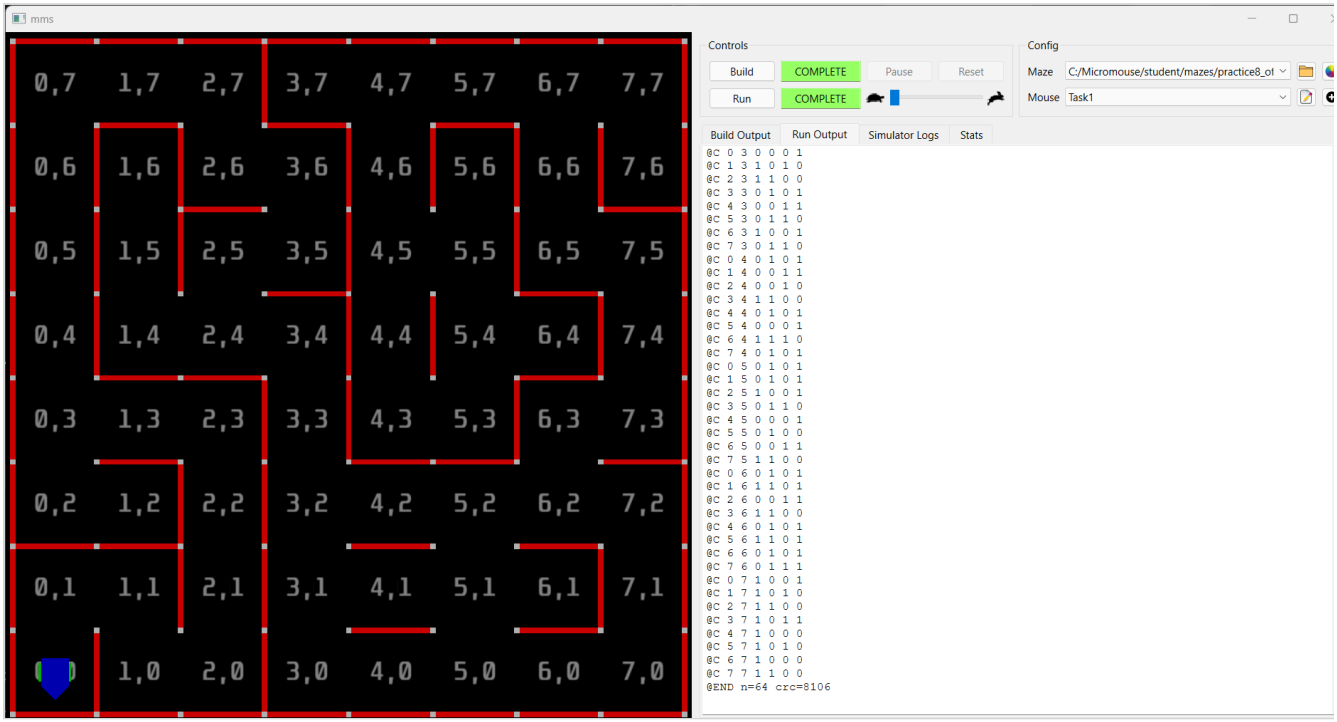


Figure 1. mms after the Task 1 run on practice8_offcentre, with the map dump in Run Output.

3.2 Map dump against the real maze

Each dump was copied from the mms Run Output and checked with mm_check.py --dump. The script verifies the CRC, prints the real maze beside a drawing of the dump, and lists every side that differs.

Maze	Size	Cells in dump	Sides known	Unknown	Wrong	Result
practice4_1	4×4	16	64	0	0	PASS
practice8_offcentre	8×8	64	256	0	0	PASS
practice16_1	16×16	256	1,024	0	0	PASS

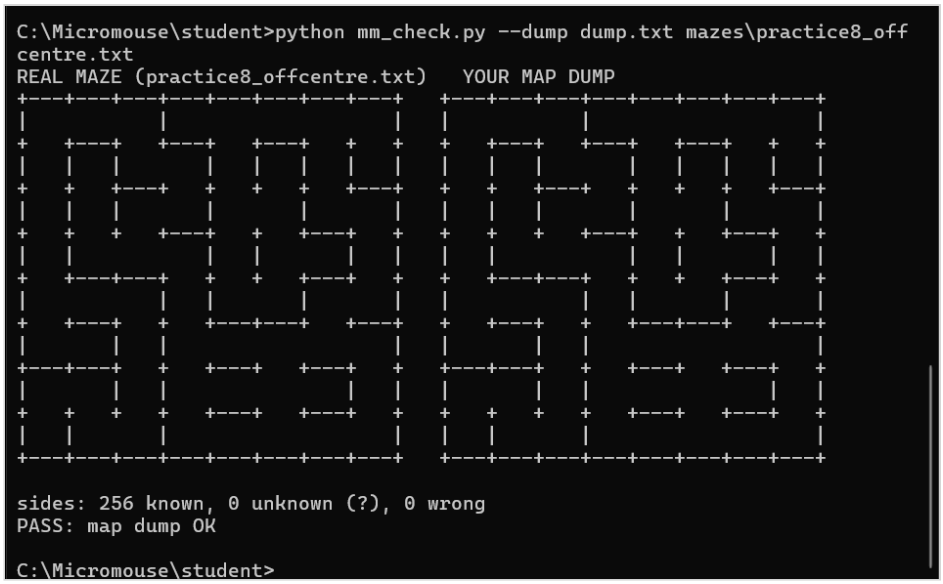


Figure 2. practice8_offcentre (8×8): real maze (left) and the team's map dump (right).

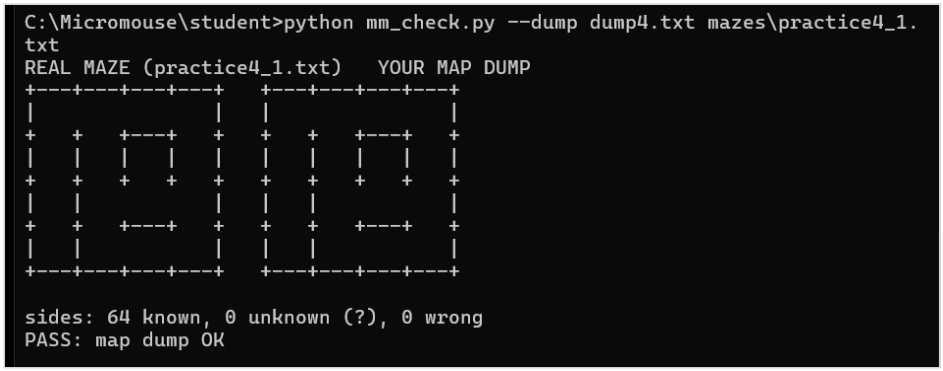


Figure 3. practice4_1 (4×4).

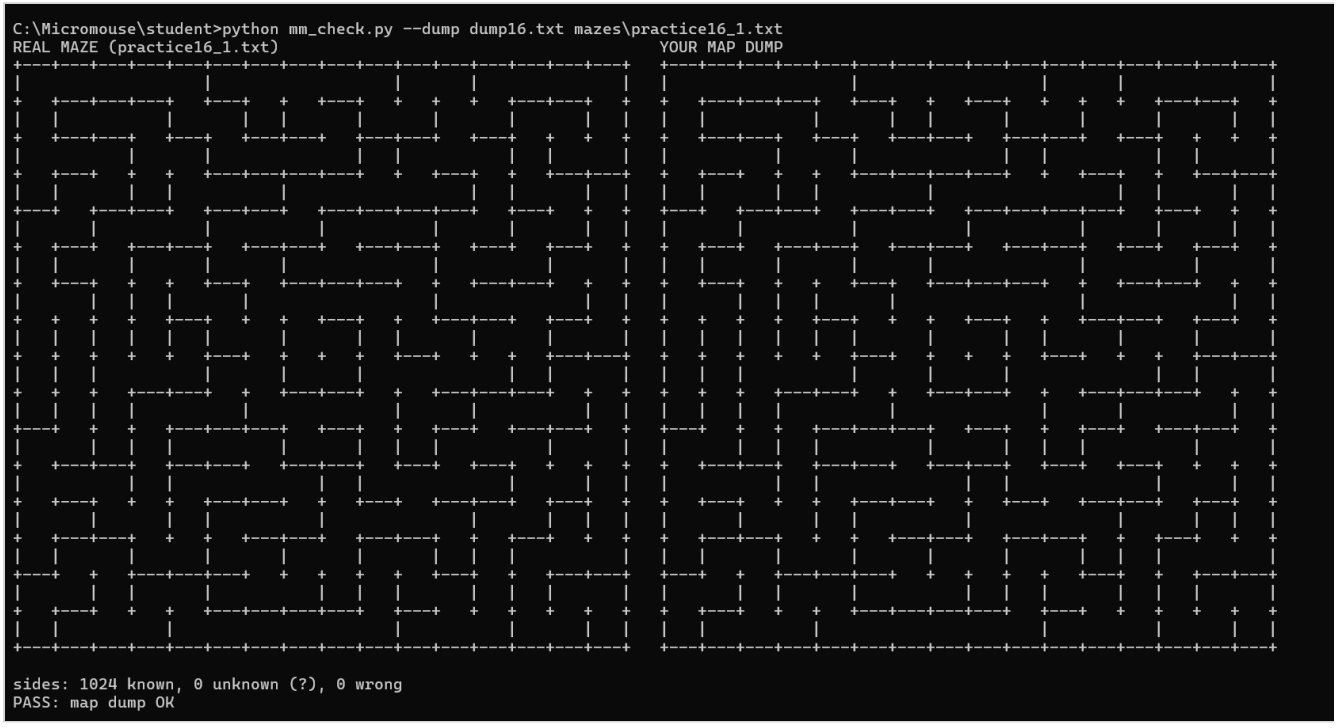


Figure 4. practice16_1 (16×16).

3.3 Goal read from mm_maze()

With practice8_offcentre.goal copied to mm_goal.txt, the HAL reported goal (6,1)-(6,1) and the solver logged the same goal. Without the file, hal_sim.c falls back to the maze centre, (3,3)-(4,4) on this 8×8 maze. The goal therefore comes from mm_maze(), not from a constant.

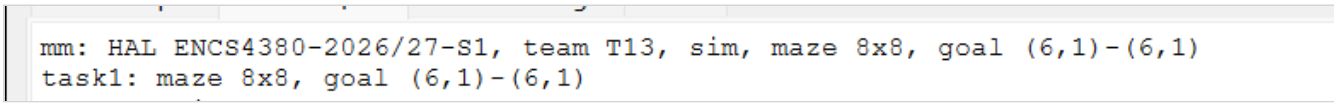


Figure 5. Run Output with the goal read from mm_maze().

4. How to reproduce

The submission folder Task1_T13 holds four files:

File	What it is
solver.c	The Task 1 solver (developed as solver_task1.c; same content)
dump4.txt	Map dump from practice4_1 (4×4), team T13
dump.txt	Map dump from practice8_offcentre (8×8), team T13
dump16.txt	Map dump from practice16_1 (16×16), team T13

To rebuild and run it, copy solver.c into the kit's student folder (here C:\Micromouse\student, with mms in C:\Micromouse\mms and gcc from MSYS2 UCRT64) and use the kit's own build command:

mms field	Value
Directory	C:\Micromouse\student
Build Command	gcc -O2 -std=c99 -o solver.exe main_sim.c hal_sim.c mm_common.c solver.c
Run Command	C:\Micromouse\student\solver.exe

1. Load a maze from mazes\ (the .txt file), press Build, then Run.
2. Copy the Run Output into a text file, e.g. dump.txt.
3. Check it against the same maze:

   ```
   python mm_check.py --dump dump.txt mazes\practice8_offcentre.txt
   ```

4. For another goal: copy mazes\practice8_offcentre.goal to mm_goal.txt before Run, and delete mm_goal.txt afterwards.

5. Tools and AI use

- mms simulator (Windows build from the course kit), gcc 16.2.0 (MSYS2 UCRT64), Python 3.15.
- mm_check.py: an offline checker that compares a map dump with a maze file. It is not the official grader.
- AI assistance: Claude (Anthropic) was used to help write and explain the Task 1 solver (solver.c) and mm_check.py, and to guide the simulator setup. The team ran every test in this report on its own PC.